

NAME: B Sandhya

REGNO: 192311292

SUBJECT: CLOUD COMPUTING AND BIG DATA ANALYTICS

QUESTION 1 – RETAIL DATA OPTIMIZATION PROBLEM

A retail chain operates physical stores and an online platform. It generates large volumes of sales, inventory, and customer behavior data. The company requires real-time inventory tracking, demand forecasting, and personalized marketing recommendations.

Proposed Big Data Architecture

Stores + Online Platform + Mobile App + Sensors → Kafka → Data Lake → Spark/Flink → ML Models → Data Warehouse → Dashboards / Recommendations

1. Data Sources and Ingestion

The system should collect data from point-of-sale terminals, online orders, inventory management systems, mobile applications, website clickstreams, customer loyalty systems, and warehouse sensors. Apache Kafka can be used as the distributed event-streaming platform. Each type of event can be published to separate topics such as sales, inventory, customer activity, and online orders.

Kafka provides high-throughput ingestion and allows multiple consumers to process the same events independently. This is important because inventory tracking, analytics, and recommendation services may need the same transaction events.

2. Distributed Storage

A cloud data lake such as Amazon S3, Azure Data Lake Storage, or Google Cloud Storage can act as the central storage layer. Hadoop HDFS is another possible distributed-storage option. Raw data should first be stored in its original form, while cleaned and curated datasets can be stored separately.

Structured sales and inventory records can be stored in relational or warehouse tables. Semi-structured clickstream and application events can be stored as JSON and later converted into efficient formats such as Parquet. Partitioning by date, store, region, and product category can reduce query time.

3. Batch and Stream Processing

Apache Spark can perform large-scale batch processing such as historical sales analysis, customer segmentation, inventory analysis, and model training. Apache Flink or Spark Structured Streaming can process live events for real-time inventory updates.

For example, when a product is sold in a physical store, the transaction is published to Kafka. A stream-processing application consumes the event and updates the inventory state. The same event can also be sent to the analytics pipeline.

4. Demand Forecasting and Personalization

Demand forecasting can use time-series and machine-learning models such as ARIMA, Prophet, Gradient Boosting, or LSTM, depending on the data and required complexity. Features may include historical sales, seasonality, promotions, holidays, price changes, and regional demand.

For personalized marketing, collaborative filtering, content-based recommendation, or hybrid recommendation models can be used. Customer browsing history, purchase history, product preferences, and responses to previous campaigns can become model features.

5. Scalability, Reliability and Security

The architecture should use horizontal scaling. Kafka partitions distribute events across brokers, while Spark/Flink distributes computation across worker nodes. Replication protects data against node failures. Cloud storage can scale as data volume increases.

Customer information should be protected using encryption, role-based access control, authentication, secure APIs, and audit logging. Sensitive customer attributes should be minimized or anonymized where possible.

6. Analytics and Dashboards

Processed data can be loaded into a cloud data warehouse or analytical database. Power BI, Tableau, or Grafana can connect to this layer to show sales, stock levels, regional demand, forecast accuracy, and campaign performance. Managers can therefore monitor the business using both historical and real-time information.

Conclusion: The proposed architecture combines Kafka, cloud data-lake storage, Spark/Flink, machine learning, analytical databases, and BI tools. It provides scalable inventory tracking, accurate forecasting, and personalized customer engagement.

QUESTION 2 – SOCIAL MEDIA FRAUD DETECTION PROBLEM

A social media platform processes billions of posts, comments, likes, and shares every day. It must detect fake accounts and misinformation quickly, requiring a scalable near-real-time big data architecture.

Proposed Architecture

Posts + Comments + Likes + Shares + Account Events → Kafka → Stream Processing → Feature Store / ML Models → Risk Score → Moderation / Alerting

1. Event Collection and Streaming

Apache Kafka can ingest user interactions at very high throughput. Separate topics can represent posts, comments, reactions, account changes, reports, and sharing activity. Kafka partitions allow the platform to distribute the workload across many brokers and consumers.

A schema registry and standardized event format can help maintain consistency as the platform evolves.

2. Stream Processing

Apache Flink or Spark Structured Streaming can analyze events as they arrive. Streaming jobs can calculate features such as posting frequency, sudden sharing bursts, repeated content, account age, unusual login patterns, and network-level interaction behavior.

For example, if a newly created group of accounts starts posting and sharing identical content at unusually high rates, the stream processor can produce a high-risk signal for further analysis.

3. Machine Learning for Fake Accounts

Fake-account detection can use supervised or semi-supervised machine-learning models. Features can include account age, activity frequency, follower-following relationships, interaction patterns, profile completeness, device or session patterns, and graph relationships.

Algorithms such as Random Forest, Gradient Boosting, Logistic Regression, or graph-based models can classify accounts. The model should produce a risk score rather than relying on a single feature.

4. Misinformation Detection

Natural Language Processing models can analyze text for suspicious or misleading patterns. Text embeddings and transformer-based classifiers can support content classification. Image or multimedia metadata can also be incorporated when relevant.

A practical system should combine automated scoring with human review for high-impact or uncertain cases. This reduces false positives and provides a mechanism for appeals and correction.

5. Storage and Historical Analytics

A data lake can store historical interaction data for model training and long-term analysis. NoSQL databases can provide fast access to user profiles and recent activity. Spark can perform large-scale historical processing and feature generation.

A feature store can maintain consistent machine-learning features for both training and real-time inference.

6. Scalability, Fault Tolerance and Security

Kafka partitions, replicated storage, distributed stream processors, checkpointing, and autoscaling allow the system to handle traffic spikes. Backpressure mechanisms can prevent downstream systems from being overwhelmed.

Security should include encryption, authentication, authorization, privacy controls, audit logs, and rate limiting. User data should be handled according to applicable privacy requirements.

7. Monitoring and Response

Dashboards can show the number of suspicious accounts, flagged content, detection rates, false-positive rates, and processing latency. Alerts can be generated for sudden fraud or misinformation campaigns.

Conclusion: Kafka combined with Flink/Spark Streaming, distributed storage, machine learning, NLP, and monitoring provides a scalable near-real-time fraud and misinformation detection platform.

QUESTION 3 – GOVERNMENT DATA PLATFORM PROBLEM

A government agency is building a national data platform that combines census, economic, and geospatial datasets. Multiple departments require shared access and interoperability while strict privacy and security must be maintained.

Proposed Architecture

Government Departments → Secure APIs / Data Integration → National Data Lake → Data Catalog + Governance → Processing / Analytics → Authorized Department Dashboards

1. Data Integration

The platform should ingest data from census systems, economic databases, geographic information systems, public-service applications, and other authorized government sources. Apache NiFi, cloud integration services, or API-based ingestion can be used.

A common data model and metadata standards should be defined so that different departments interpret fields consistently.

2. Storage and Data Organization

A secure government data lake can store structured, semi-structured, and geospatial datasets. Structured datasets can be maintained in relational or analytical databases, while large files can be stored in distributed object storage.

Data should be organized into raw, validated, and curated zones. Partitioning and indexing can improve query performance. Geospatial datasets can be stored using spatial databases such as PostGIS.

3. Interoperability

Interoperability requires common formats, metadata, identifiers, APIs, and agreed data definitions. REST APIs and standardized exchange formats such as JSON or CSV can support data sharing. A central data catalog should describe dataset ownership, meaning, quality, update frequency, and access conditions.

A master-data strategy can help ensure that important entities such as geographic regions and administrative units are represented consistently across departments.

4. Data Governance

A formal governance framework should define data ownership, stewardship, quality standards, retention periods, classification, sharing rules, and accountability. Data quality checks should detect missing, duplicate, inconsistent, or outdated records.

The data catalog should maintain lineage information showing where data came from, how it was transformed, and which departments use it.

5. Privacy and Security

Sensitive citizen information should be classified and protected using encryption at rest and in transit. Role-based or attribute-based access control can ensure that departments only access authorized datasets.

Strong authentication, audit logs, key management, secure APIs, network segmentation, and continuous monitoring should be implemented. Personally identifiable information should be masked, anonymized, or pseudonymized when full identity is not required for analysis.

6. Analytics and Access

Spark or cloud analytics services can process national-scale datasets. Departments can use controlled analytical workspaces and BI dashboards. Access should follow the principle of least privilege.

Conclusion: A national government data platform should combine a secure data lake, interoperability standards, cataloging, governance, distributed analytics, and strong privacy controls. This enables data sharing without sacrificing security or accountability.

QUESTION 4 – BANKING FRAUD DETECTION PROBLEM

A bank processes millions of transactions every minute and must identify fraudulent activity with low latency and high accuracy. The system must combine historical transaction data with real-time transaction streams.

Proposed Fraud Detection Pipeline

ATM / Mobile / Card / Online Transactions → Kafka → Stream Processor → Feature Store + ML Model → Fraud Score → Approve / Review / Block

1. Real-Time Data Ingestion

Apache Kafka can ingest transaction events from card systems, ATMs, mobile banking, internet banking, and payment gateways. Transactions can be partitioned by account, region, or transaction stream to distribute processing.

The streaming layer must preserve event ordering where required and provide reliable delivery so that transactions are not lost.

2. Stream Processing and Feature Generation

Apache Flink or Spark Structured Streaming can calculate real-time features such as transaction frequency, amount deviation, geographic distance, device changes, merchant patterns, and recent failed attempts.

For example, a transaction that differs greatly from a customer's normal behavior can receive a higher risk score. Sliding and tumbling windows can be used to analyze activity over recent time periods.

3. Machine Learning Algorithms

Supervised models such as Logistic Regression, Random Forest, Gradient Boosting, or XGBoost can be trained using historical labeled transactions. For previously unseen fraud patterns, anomaly-detection methods such as Isolation Forest or autoencoders can provide additional signals.

A hybrid approach is preferable because fraudulent transactions are usually rare and patterns change over time. The final decision can combine machine-learning scores with business rules.

4. Historical Data and Model Training

Historical transactions can be stored in a secure data lake or warehouse. Spark can process large datasets for feature engineering and model training. The training pipeline should include class-imbalance handling, validation, model evaluation, and periodic retraining.

Metrics such as precision, recall, F1-score, false-positive rate, and detection latency are important because accuracy alone can be misleading for highly imbalanced fraud datasets.

5. Low Latency and Scalability

The real-time path should contain only the processing required for an immediate decision. Frequently used customer features can be kept in a low-latency feature store or cache such as Redis. Kafka partitions and distributed stream processors allow the system to scale as transaction volume grows.

Checkpointing, replication, retries, and fault-tolerant deployment ensure continued processing during failures.

6. Security and Monitoring

Financial data must be protected through encryption, strong authentication, authorization, secure key management, tokenization where applicable, audit logging, and network security. Access to transaction data should be strictly controlled.

Operational dashboards should monitor fraud alerts, detection latency, transaction volumes, model performance, and false positives.

Conclusion

The recommended solution combines Kafka, Flink/Spark Streaming, secure historical storage, low-latency feature access, machine learning, and business rules. This architecture can provide fast fraud detection while balancing accuracy, scalability, and operational reliability.

QUESTION 5 – AGRICULTURE SMART FARMING PROBLEM

A smart farming system collects soil, weather, crop, sensor, and drone data from distributed agricultural fields. Farmers require actionable insights to improve crop yield and resource utilization.

Proposed Precision Agriculture Architecture

Soil Sensors + Weather Stations + Drones + Satellite/Data Sources → Edge Gateway → Kafka/IoT Platform → Data Lake → Spark/Flink → ML + Analytics → Farmer Dashboard / Automated Actions

1. Data Collection

IoT sensors can measure soil moisture, temperature, pH, humidity, nutrient-related measurements, and other field conditions. Weather stations provide rainfall, temperature, wind, and humidity. Drones can collect images for crop health and field inspection.

Because farms are geographically distributed, edge gateways can aggregate and temporarily store data before sending it to the cloud. This helps when network connectivity is limited.

2. Data Ingestion and Storage

An IoT messaging platform or Apache Kafka can ingest sensor events. A cloud data lake can store historical sensor readings and drone images. Time-series databases are suitable for frequent sensor measurements, while object storage is suitable for large drone images.

Data should be partitioned by farm, field, sensor type, and date to support efficient analysis.

3. Real-Time Stream Processing

Flink or Spark Structured Streaming can analyze sensor readings continuously. For example, if soil moisture falls below an acceptable level, the system can generate an irrigation recommendation or trigger an authorized automated irrigation system.

Real-time weather changes can also be combined with sensor readings to improve irrigation and crop-protection decisions.

4. Machine Learning and Analytics

Historical data can be used to build yield-prediction models using algorithms such as Random Forest, Gradient Boosting, or other regression models. Features may include soil conditions, weather, crop variety, planting date, irrigation, and historical yield.

Drone images can be analyzed using computer-vision models to identify possible crop stress, disease symptoms, weed presence, or uneven growth. These results can be combined with sensor data for field-level recommendations.

5. Resource Optimization

Analytics can support precision irrigation by identifying areas that require more or less water. Fertilizer application can also be optimized using soil and crop-condition data. This can reduce unnecessary resource usage while supporting productivity.

Farmers can receive recommendations through dashboards or mobile applications showing field maps, moisture levels, weather forecasts, crop-health indicators, and predicted yield.

6. Scalability, Reliability and Security

Distributed cloud storage and processing allow additional farms and sensors to be added without redesigning the whole system. Kafka partitions and distributed processing nodes can handle increasing event volumes.

Device authentication, encrypted communication, access control, secure APIs, and regular backups should protect farm and sensor data. Edge devices should also be securely configured and monitored.

7. Decision-Making Dashboard

A dashboard can combine real-time and historical information. Farmers can compare current soil moisture with previous measurements, inspect drone imagery, review weather conditions, and see predicted crop yield. This converts raw sensor data into practical decisions.

Conclusion: Precision agriculture benefits from IoT/edge computing, Kafka or IoT messaging, cloud data lakes, Spark/Flink, time-series storage, machine learning, and visualization. The solution improves yield prediction and helps farmers use water, fertilizer, and other resources more efficiently.